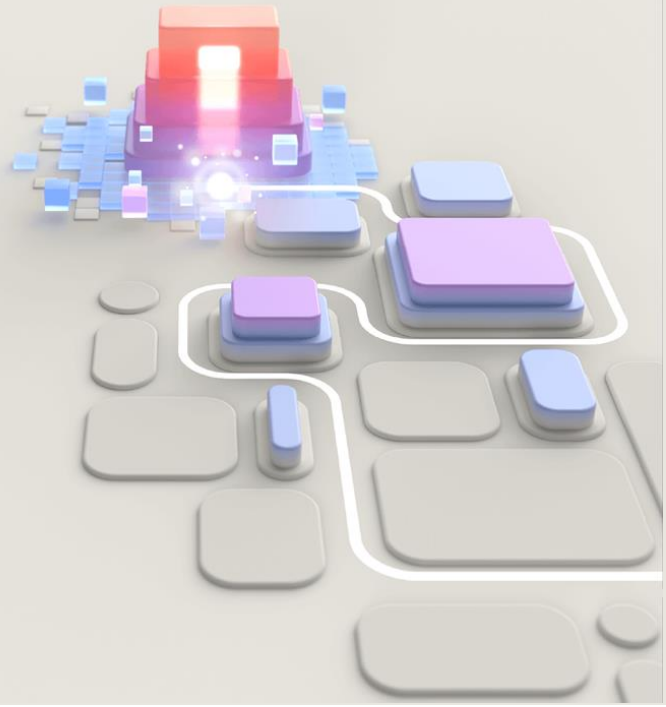
Microsoft

GitHub Foundations [GH-900]



© Copyright Microsoft Corporation. All rights reserved.

1

Let's have a great time together

We all contribute to a great class

- This placeholder is designed with bulleted text
- It's easy to remove or add bullets
- Be on time (class hours)
- Be present (avoid mail, calls, side meetings)

What you should know about our facilities

- This placeholder is designed with bulleted text
- This section can be removed if the class is online
- Accommodations
- Internet access
- Parking
- Restrooms
- Emergency

© Copyright Microsoft Corporation. All rights reserved.

2

About this course

Prerequisites

- Familiarity with integrated development environments (IDEs) and version control systems like GitHub.
- Foundational knowledge of AI and machine learning principles.

Audience profile

- Software developers who want to learn about version control, Git, and GitHub.
- Documentation writers who collaborate on various projects and need to manage changes effectively.
- Teams that work on collaborative projects and need to track changes, manage project history, and facilitate collaboration will benefit from this course.
- Individuals who are new to Git and GitHub and want to gain a solid understanding of their fundamental features and collaborative tools are the primary audience.

© Copyright Microsoft Corporation. All rights reserved.

3

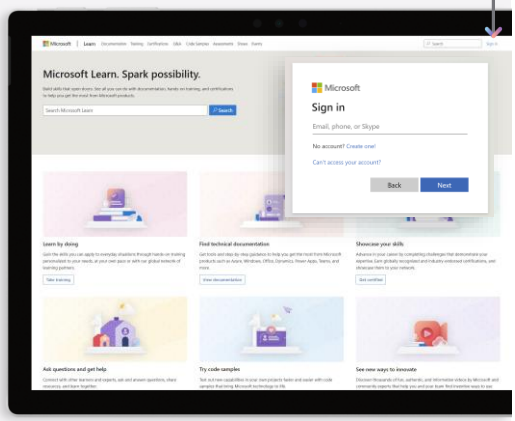
Get the most out of your Microsoft Learn profile

Verify, track, and share your training and certification progress and accomplishments—*all on one platform*

- Claim your achievement code for this course and share you have completed it.
- Access your course material and track progress on your learning activities.
- Share and verify your Microsoft Certifications via email, on social networking platforms, and on your résumé.
- Download and print transcripts and certificates.
- Manage your upcoming activities and certification exam appointments.

www.aka.ms/MyMicrosoftLearnProfile

Create your Microsoft Learn profile at learn.microsoft.com



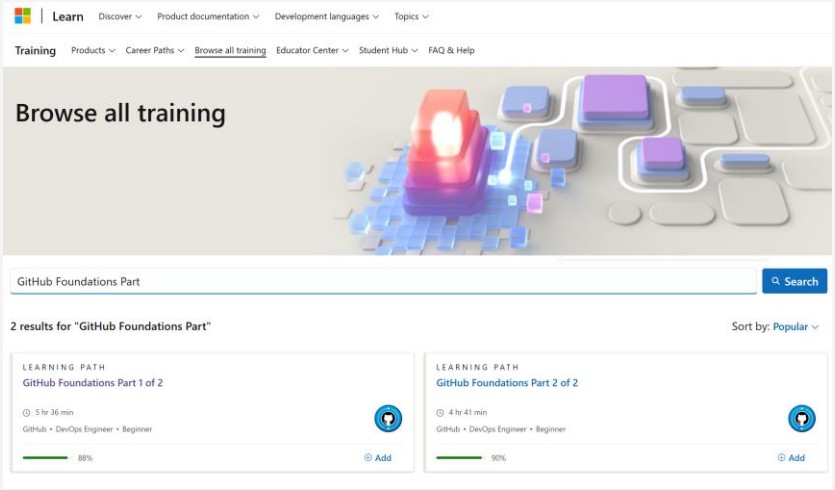
© Copyright Microsoft Corporation. All rights reserved.

4

© Microsoft Corporation

Access your course materials

All course content is available on Microsoft Learn



© Copyright Microsoft Corporation. All rights reserved.

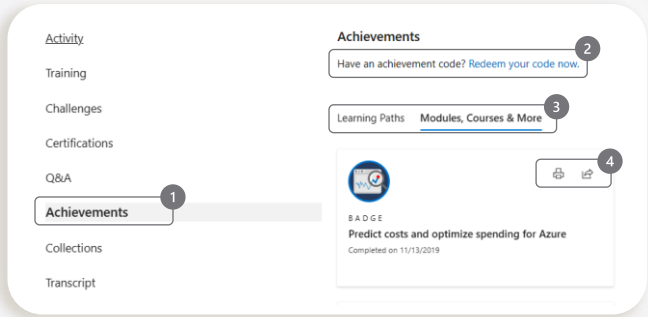
- We'll go through this content together and as the course progresses, I will advise you on which modules to review.
- You can provide feedback for modules on Microsoft Learn. Use the link at the bottom of the page.
- Detailed lab instructions are included in your lab environment.

5

Celebrate your accomplishments and feel empowered

Get your achievement recognized and earn your badge for completing this course

aka.ms/MyMicrosoftLearnProfile



1. Go to **Achievements** on your Microsoft Learn profile
2. Redeem the code provided by your trainer
3. Find your new badge **on Modules, Courses & More**
4. Share your new achievement with your professional network. You can also download and print your certificate.

6

© Microsoft Corporation

Get ready for your Microsoft Certification exam

You can validate your code security knowledge with the GitHub Foundations certificate.

Domain Breakdown	Exam Percentage
Domain 1: Introduction to Git and GitHub	22%
Domain 2: Working with GitHub Repositories	8%
Domain 3: Collaboration Features	30%
Domain 4: Modern Development	13%
Domain 5: Project Management	7%
Domain 6: Privacy, Security, and Administration	10%
Domain 7: Benefits of the GitHub Community	10%



© Copyright Microsoft Corporation. All rights reserved.

7

Module 1: Introduction to Git



© Copyright Microsoft Corporation. All rights reserved.

8

What is version control?

A version control system (VCS) is a program or set of programs that tracks changes to a collection of files. VCS is also known as software configuration management (SCM) system.

With a VCS, you can:

- View changes made to your project, when, and who made them.
- Include a message with each change and an explain.
- Retrieve past versions of the entire project or of individual files.
- Create branches, where changes can be made experimentally. This allows several different sets of changes to be worked on at the same time. Later, you can merge the changes you want to keep back into the main branch.
- Attach a tag to a version—for example, to mark a new release.

© Copyright Microsoft Corporation. All rights reserved.

9

What is version control? (terminology)

- **Working tree:** The set of nested directories and files that contain the project.
- **Repository (repo):** The directory, located at the top level of a working tree, where Git keeps all the history and metadata for a project.
- **Hash:** A number produced by a hash function that represents the contents of a file or another object as a fixed number of digits.
- **Object:** A Git repo contains four types of objects:
  - *blob* object contains an ordinary file, *tree* object represents a directory, *commit* object represents a specific version of the working tree, and *tag* is a name attached to a commit.
- **Commit:** When used as a verb, commit means to make a commit object.
- **Branch:** A branch is a named series of linked commits. The default branch, which is created when you initialize a repository, is called main, often named master in Git.

© Copyright Microsoft Corporation. All rights reserved.

10

© Microsoft Corporation

Basic Git commands

git status

`git status` displays the state of the working tree (and of the staging area—we'll talk more about the staging area soon). It lets you see which changes are currently being tracked by Git, so you can decide whether you want to ask Git to take another snapshot.

git add

`git add` is the command you use to tell Git to start keeping track of changes in certain files.

The technical term is *staging* these changes. You'll use git add to stage changes to prepare for a commit. All changes in files that have been added but not yet committed are stored in the *staging area*.

git commit

`git commit` has the same meaning as when you commit to a plan or commit a change to a database. As a verb, committing changes means you put a copy of the file or directory in the repository as a new version. As a noun, a commit is the small chunk of data that gives the changes you committed a unique identity.

© Copyright Microsoft Corporation. All rights reserved.

11

Basic Git commands (cont.)

git log

The `git log` command allows you to see information about previous commits. Each commit has a message attached to it (a commit message), and the git log command prints information about the most recent commits, like their time stamp, the author, and a commit message. This command helps you keep track of what you've been doing and what changes have been saved.

git help

Use `git help` to easily get information about all the commands you've learned so far, and more.

Remember, each command comes with its *own* help page, too. You can find these help pages by typing `git <command> --help`.

For example, `git commit --help` brings up a page that tells you more about the git commit command and how to use it.

© Copyright Microsoft Corporation. All rights reserved.

12

© Microsoft Corporation

Module 2: Introduction to GitHub



© Copyright Microsoft Corporation. All rights reserved.

13

What is GitHub?



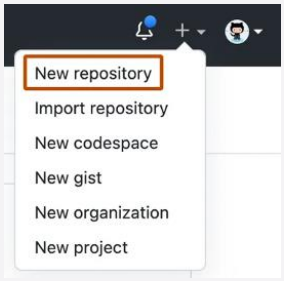
© Copyright Microsoft Corporation. All rights reserved.

14

What is GitHub? (Introduction to repositories)

Public repositories are accessible to everyone on the internet.

Private repositories are only accessible to you, people you explicitly share access with, and, for organization repositories, certain organization members.





© Copyright Microsoft Corporation. All rights reserved.

15

Components of the GitHub flow



Branches

- Make changes without affecting the entire project.
- Safe place to experiment with new features or fixes.
- Revert changes or push more changes.
- Changes won't update on the default branch until you merge your branch.

© Copyright Microsoft Corporation. All rights reserved.

16

© Microsoft Corporation

Components of the GitHub flow



Commits

- A change to one or more files on a branch.
- Every time a commit is created, it's assigned a unique ID and tracked along with the time and contributor.
- Commits provide an audit trail for reviewing the history of a file or linked item.
- The primary states for a file in a Git repository are **Untracked** and **Tracked**.

© Copyright Microsoft Corporation. All rights reserved.

17

Components of the GitHub flow



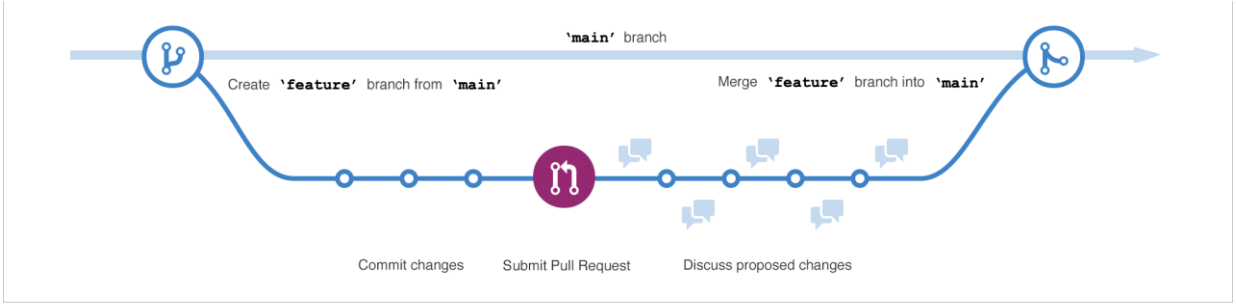
Pull requests

- Mechanism used to signal that the commits from one branch are ready to be merged into another branch.
- The team member submitting the pull request asks one or more reviewers to verify the code and approve the merge.
- Reviewers can comment on changes, add their own, or use the pull request itself for further discussion.

© Copyright Microsoft Corporation. All rights reserved.

18

Components of the GitHub flow



© Copyright Microsoft Corporation. All rights reserved.

19

GitHub is a collaborative platform



Creating an Issue:

- Navigate to the repository's main page, select "Issues," then "New issue."
- Use templates or open a blank issue.
- Enter a title and description, and optionally assign, label, or add to a project.
- Submit the new issue.

Using GitHub Discussions:

- Discussions are for open, accessible conversations not related to code.
- Enable discussions using repository settings, configure a welcome post, and start discussions.
- Discussions can be created by any user who can view the repository.

Enabling Discussions:

- Repository owners or those with Write access can enable discussions.
- Navigate to repository settings, scroll to Features, and select "Setup discussions."
- Edit the template for the welcome post and start the discussion.

© Copyright Microsoft Corporation. All rights reserved.

20

© Microsoft Corporation

GitHub platform management



Branches

- You can subscribe to notifications for specific issues, pull requests, gists, CI activity, repository events, and more.
- Automatic subscriptions occur when you open or comment on issues/pull requests, or are assigned to them.
- You can unsubscribe or customize notifications as needed.

© Copyright Microsoft Corporation. All rights reserved.

21

GitHub platform management



Commits

- Notifications are updates for activities you're subscribed to.
- You can use qualifiers like mentions: to find issues mentioning a specific user.

© Copyright Microsoft Corporation. All rights reserved.

22

GitHub platform management



GitHub Pages

- GitHub Pages allows you to host a static website directly from a GitHub repository.
- It supports HTML, CSS, and JavaScript, and can include a build process for publishing.

© Copyright Microsoft Corporation. All rights reserved.

23

Exercise - A guided tour of GitHub



Tour guides you through everything you need to start using GitHub in less than an hour.

This exercise checks your knowledge of GitHub key features:

- Committing a branch
- Committing a file
- Opening a pull request
- Merging a pull request

Start course

© Copyright Microsoft Corporation. All rights reserved.

24

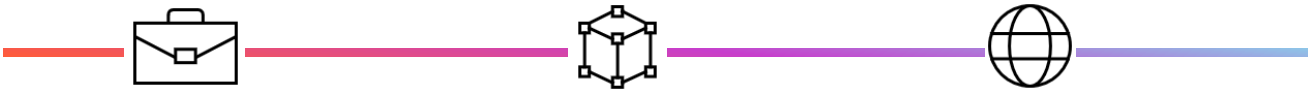
Module 3: Introduction to GitHub's products



© Copyright Microsoft Corporation. All rights reserved.

25

GitHub accounts and plans



Personal Accounts

- Personal accounts are individual user identities on GitHub, owning resources like repositories.
- Actions taken on GitHub are attributed to personal accounts.
- Personal accounts can own unlimited public and private repositories with collaborators.

Organizational Accounts

- Allow multiple people to collaborate on projects with tiered permissions.
- Organizations own resources, but actions are attributed to individual personal accounts.
- Members have different roles, with owners and security managers controlling settings and access.

Enterprise Accounts

- Manage policies and billing for multiple organizations centrally.
- Enterprise owners can invite or transfer organizations and create new ones.
- Policies set at the enterprise level control options available to all owned organizations.

© Copyright Microsoft Corporation. All rights reserved.

26

GitHub accounts and plans (cont.)

GitHub Free

- Provides basic features for individuals and organizations,
- Personal accounts get community support, Dependabot alerts, 500-MB Packages storage, 120 Codespaces core hours.
- Organizations get team access controls and unlimited collaborators on public repositories.

GitHub Pro

- Designed for individual developers needing advanced tools and insights.
- Includes all GitHub Free features plus email support, 3,000 Actions minutes, 2-GB Packages storage, 180 Codespaces core hours, and advanced repository tools.

GitHub Team

- Enhanced version of GitHub Pro for organizations with increased Actions minutes and Packages storage.
- Offers email support, advanced repository tools, draft pull requests, team pull request reviewers, and scheduled reminders.

GitHub Enterprise

- Provides advanced support, security, compliance, and deployment controls for multiple organizations.
- Includes all GitHub Team features plus SAML single sign-on, SCIM provisioning, and GitHub Advanced Security.
- Available as GitHub Enterprise Server (self-hosted) or GitHub Enterprise Cloud (cloud-hosted).

© Copyright Microsoft Corporation. All rights reserved.

27

GitHub Mobile and GitHub Desktop

GitHub Mobile

- Manage, triage, and clear notifications from GitHub.com.
- Read, review, and collaborate on issues and pull requests.
- Edit files in pull requests and interact with users, repositories, and organizations.
- Receive push notifications for mentions, schedule custom notification hours, and secure your account with two-factor authentication.

GitHub Desktop

- Add and clone repositories, and add changes to your commit interactively.
- Quickly add coauthors to your commit and check out branches with pull requests.
- View continuous integration (CI) statuses and compare changed images.
- Facilitate team collaboration and share Git and GitHub best practices.

© Copyright Microsoft Corporation. All rights reserved.

28

GitHub billing

GitHub bills separately for each account. You receive a separate bill for your personal account and for each organization or enterprise account you own.

The bill for each account is a combination of charges for your subscriptions and usage-based billing.

- **Subscriptions** include your account's plan, such as GitHub Pro or GitHub Team, and paid products that have a consistent monthly cost, such as GitHub Copilot and apps from GitHub Marketplace.
- **Usage-based billing** applies when the cost of a paid product depends on how much you use the product. For example, the cost of GitHub Actions depends on how many minutes your jobs spend running and how much storage your artifacts use.

29

Module 4: Configure code scanning on GitHub



© Copyright Microsoft Corporation. All rights reserved.

30

What is code scanning?





Code Scanning with CodeQL

- **Purpose:** Analyzes code to find security vulnerabilities and coding errors, displaying alerts in the repository's Security tab.
- Default setup for quick configuration, advanced setup for customizable workflows, and running CodeQL CLI.
- Includes C/C++, C#, Go, Java/Kotlin, JavaScript/TypeScript, Python, Ruby, and Swift.

© Copyright Microsoft Corporation. All rights reserved.

Enabling CodeQL

- **Setup:** Navigate to the repository's Security tab, select Set up code scanning, and choose Default to enable CodeQL.
- **Triggers:** Code scanning runs automatically on push to protected branches or pull requests against the default branch.
- **Advanced Setup:** Generates a workflow file for more advanced configuration and additional steps.

Billing for Actions

- **Usage:** Code scanning uses GitHub Actions, consuming minutes for each run.
- **Free Usage:** Free for public repositories and self-hosted runners; private repositories have limited free minutes and storage.
- **Spending Limits:** Monthly billed accounts have a default spending limit of 0 USD, while invoiced accounts have unlimited spending limits.

31

Enable code scanning with third party tools

SARIF File Uploads for Code Scanning

- **Purpose:** Allows uploading of SARIF files generated outside GitHub to display code scanning alerts in the repository.
- Upload results using the code-scanning API, CodeQL CLI, or GitHub Actions.

Code-Scanning API

- **Functionality:** Retrieve and update information on code scanning alerts, analyses, and configurations.
- Access via HTTPS, using custom media types like application/sarif+json for specific endpoints.

CodeQL CLI

- Purpose: Analyze code and generate SARIF files for upload to GitHub.
- Components: Includes the CodeQL CLI product, queries, libraries, and precompiled queries for compatibility and performance.

GitHub Actions for SARIF Uploads

- Workflow: Use the upload-sarif action in a GitHub Actions workflow to upload SARIF files.
- Configuration: Set input parameters like sarif-file and configure the action to run on push or scheduled events.

32

© Microsoft Corporation

Configure code scanning

Configuring Code Scanning

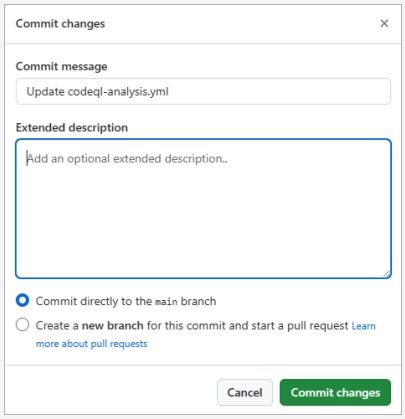
- Choose your own configuration to save time and set the best frequency for scans.

Editing Code-Scanning Workflow

- Workflow files are stored in the `.github/workflows` directory.
- Common Edits: Adjust scan frequency, specify languages or directories, and configure what CodeQL looks for in your code.

Scan Triggers

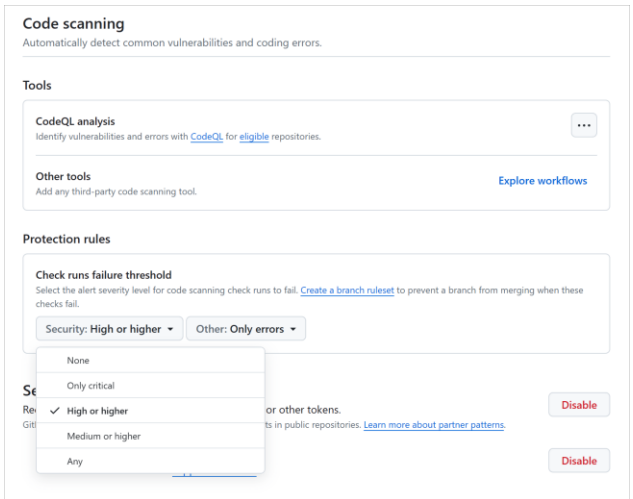
- Push Events: Default setup triggers scans on every push to the default or protected branches.
- Pull Requests: Scans are triggered on pull requests targeted against the default branch, with results appearing as alerts in pull-request checks.



33

Configure code scanning (cont.)

- **Pull Request Check Failures:** Customize alert severity levels that trigger pull request check failures in the repository settings under *Code security and analysis*.
- **Avoiding Unnecessary Scans:** Use `paths-ignore` in the code-scanning workflow to prevent scans on specific file changes, and adjust the `cron` value to modify the scanning schedule.



© Copyright Microsoft Corporation. All rights reserved.

34

© Microsoft Corporation

Exercise - Configure code scanning



This exercise checks your knowledge on configuring code scanning for your repository.

This exercise walks you through the following steps:

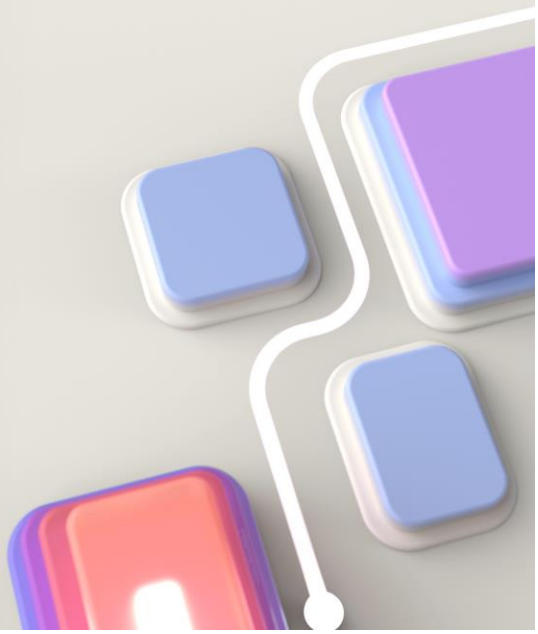
- How to enable code scanning and identify SQL injection vulnerabilities with CodeQL.
- How to build a secure software development pipeline that allows you to identify and prevent new security vulnerabilities from being introduced into your production code.

Start course

© Copyright Microsoft Corporation. All rights reserved.

35

Module 5: Introduction to GitHub Copilot

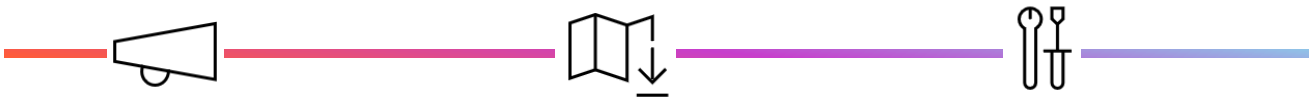


© Copyright Microsoft Corporation. All rights reserved.

36

© Microsoft Corporation

GitHub Copilot, your AI pair programmer



Copilot for Chat

- GitHub Copilot integrates a ChatGPT-like chat interface into VS Code and Visual Studio.
- It recognizes typed code and error messages, providing in-depth analysis and explanations.
- Developers can generate unit tests and receive proposed fixes for bugs.

© Copyright Microsoft Corporation. All rights reserved.

Copilot for Pull Requests

- Utilizes OpenAI's GPT-4 model to add AI-powered tags in pull-request descriptions.
- Automatically fills out tags based on changed code, which developers can review or modify.
- Available through a GitHub app that can be installed by organization admins and repository owners.

Copilot for the CLI

- Assists developers in composing commands and loops in the terminal.
- Helps with remembering precise syntax and using obscure find flags.
- Enhances productivity by reducing the need to scroll through pages for command syntax.

37

Interact with Copilot

Inline suggestions
- To accept a suggestion, select the **Tab** key or the **>** (right arrow) key.
- To reject a suggestion, keep typing or select the **Esc** key.

Command palette
1. Open the command palette by selecting **Ctrl+Shift+P** (Windows or Linux) or **Cmd+Shift+P** (Mac).
2. Enter Copilot to see available commands.
3. Select actions like **Explain This** or **Generate Unit Tests** to get assistance.

Copilot chat
1. Open the Copilot chat panel in your IDE.
2. Enter questions or requests in natural language, and then evaluate the Copilot response.

Comments to code
When you select the **Enter** key, Copilot generates code based on your description.

```
// Function to reverse a string
def reverse_string(s):
    return s[::-1]
```

38

© Microsoft Corporation

Set up, configure, and troubleshoot GitHub Copilot

Signing Up for GitHub Copilot

- Set up a free trial or subscription for your account via GitHub settings under "Code, planning, and automation."
- Install an extension for your preferred environment (supports GitHub.com, VS Code, Visual Studio, JetBrains IDEs, and Neovim).

Configuring GitHub Copilot in VS Code

- Install the GitHub Copilot extension from the Visual Studio Marketplace and open VS Code.
- Sign in to GitHub in VS Code if not previously authorized.
- Enable or disable GitHub Copilot and configure advanced settings within VS Code for code autocompletion.

© Copyright Microsoft Corporation. All rights reserved.

39

Exercise - Develop with AI-powered code suggestions by using GitHub Copilot and VS Code



GitHub Copilot helps by offering autocomplete-style suggestions.

Learn how GitHub Copilot works, and what to consider while using GitHub Copilot.

This exercise walks you through the following steps:

- Install GitHub Copilot by using GitHub Codespaces
- Prompt GitHub Copilot for code suggestions
- Accept code suggestions from GitHub Copilot

Start course

© Copyright Microsoft Corporation. All rights reserved.

40

Module 6: Code with GitHub Codespaces

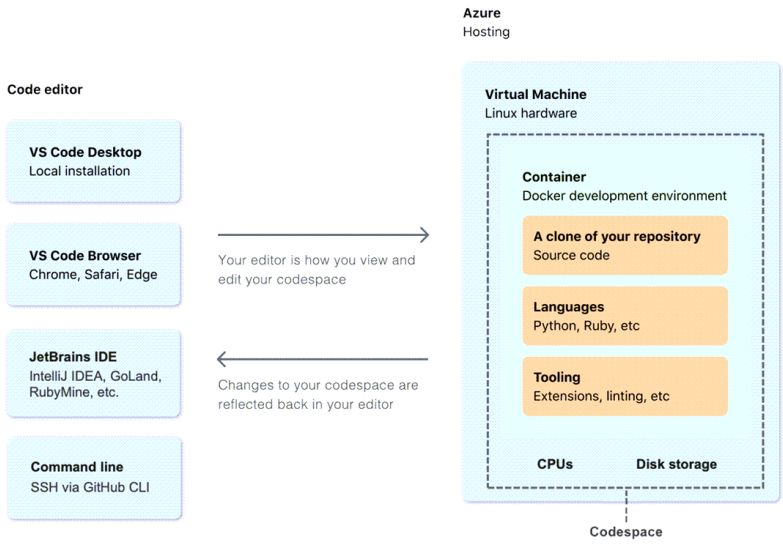
© Copyright Microsoft Corporation. All rights reserved.

41

The Codespace lifecycle

There are four ways to create a Codespace:

- From a GitHub template GitHub.com to start a new project.
- From a branch in your repository.
- From an open pull request, to explore work-in-progress.
- From a commit in a repository's history to investigate a bug.



© Copyright Microsoft Corporation. All rights reserved.

42

© Microsoft Corporation

Personalize your Codespace

Settings Sync and Dotfiles

- Synchronize VS Code settings between the desktop application and the web client.
- Use a dotfiles repository to specify scripts, shell preferences, and other configurations.

Codespace Customization

- Rename Codespaces to differentiate between them.
- Change the shell in a Codespace, open new terminal windows with a preferred shell, and configure shells using dotfiles.

Machine Type and Editor Settings

- Change the machine type running your Codespace to match your resource needs.
- Set your default editor for Codespaces in your personal settings, with options including VS Code, JetBrains Gateway, and JupyterLab.

Region and Timeout Settings

- Set your default region in the GitHub Codespaces profile settings to personalize data location.
- Specify a default timeout period for Codespaces, with a default of 30 minutes of inactivity.

43

Codespaces versus GitHub.dev editor



Use GitHub.dev to navigate files and sources code repositories from GitHub, and make and commit code changes.

Open any repository, fork, or pull request in GitHub.dev editor.

	GitHub.dev	GitHub Codespaces
Cost	Free	Free monthly quota of usage for personal accounts.
Availability	Available to everyone on GitHub.com	Available to everyone on GitHub.com.
Startup	GitHub.dev opens instantly with a key-press and you can start using it right away without having to wait for configuration or installation.	When you create or resume a Codespace, the Codespace is assigned a VM. The container is then configured based on the contents of a devcontainer.json file. This setup takes a few minutes to create the development environment.
Compute	There are no associated compute resources, so you can't build and run your code or use the integrated terminal.	With GitHub Codespaces, you get the power of a dedicated VM to run and debug your application.
Terminal access	None	GitHub Codespaces provides a common set of tools by default, meaning that you can use the Terminal exactly as you would in your local environment.
Extensions	Only a subset of extensions that can run on the web appear in the extensions view and can be installed	With GitHub Codespaces, you can use most extensions from the Visual Studio Code Marketplace.

© Copyright Microsoft Corporation. All rights reserved.

44

© Microsoft Corporation

Exercise - Code with Codespaces and Visual Studio Code



Now that you understand the Codespaces lifecycle and processes, it's time to practice coding in Codespaces and Visual Studio Code (VS Code).

This exercise walks you through the following steps:

- Create your first codespace
- Add a custom image
- Customize your codespace
- Personalize your codespace

Start course

© Copyright Microsoft Corporation. All rights reserved.

45

Module 7: Manage your work with GitHub Projects



© Copyright Microsoft Corporation. All rights reserved.

46

Projects versus Projects Classic

let's take a moment to walk through what has changed from Projects (Classic).

	Projects	Projects (Classic)
Tables and Boards	Boards, Lists, Timeline Layout	Boards
Data	Sort, rank, and group items by custom fields such as text, number, date, iteration and single select	Columns and Cards
Insights	Create visuals to help understand your work through historical and current charts with Projects	Progress bar
Automation	Use GraphQL API, Actions, and Column presets to manage your Project	Configure Column presets for when issues and pull requests are added, edited, or closed

© Copyright Microsoft Corporation. All rights reserved.

47

Insight and automation with projects

You can create charts to show the number of items assigned to each individual, or the number of issues assigned to each upcoming iteration.

Open any repository, fork, or pull request in GitHub.dev editor.



© Copyright Microsoft Corporation. All rights reserved.

48

© Microsoft Corporation

Insight and automation with projects (cont.)

Historical charts are time-based charts that allow you to view your Project's trends and progress.

View the number of items over time grouped by status and other fields.

The Burn up chart shows item status over time, allowing you to visualize progress and spot patterns.



© Copyright Microsoft Corporation. All rights reserved.

49

Module 8: Communicate effectively on GitHub using Markdown



© Copyright Microsoft Corporation. All rights reserved.

50

Markdown

A markup language, Markdown offers a lean approach to content editing. It defines a concise, lightweight syntax that strips out the overhead inherent to HTML, providing a more approachable creation experience.

it's become the standard for sites like GitHub with broad editor support in both client and browser forms.

By the end of this module, you'll be able to:

- Use Markdown to add lists, images, and links in a comment or text file.
- Determine where and how to use Markdown in a GitHub repository.
- Learn about syntax extensions available in GitHub (GitHub-flavored Markdown).

51

Markdown

Emphasize Text
Use single asterisks (*) or underscores (_) for italics, and double asterisks (**) or underscores (__) for bold text.

Declare Headings
Use the # symbol for headings, with one # for each heading level from 1 to 6.

Link to Images and Sites
Use the syntax !alt text for images and link text for site links.

Make Lists
Create ordered lists with numbers and unordered lists with asterisks (*) or dashes (-), including nested items through indentation.

Build Tables
Construct tables using pipes (|) for column breaks and dashes (-) to designate headers.

Work with Code
Use backticks (`) for inline code and triple backticks (```) for multi-line code blocks, with optional syntax highlighting.

Cross-Link Issues and Pull Requests
Link to issues and pull requests using the format #ID, such as #3602, or other conventions like GH-3602.

52

© Microsoft Corporation

Exercise - Communicate using Markdown



You will walk you through creating and using headings more effectively, organizing thoughts in bulleted lists, and showing how much work you've completed with checklists.

This exercise walks you through the following steps:

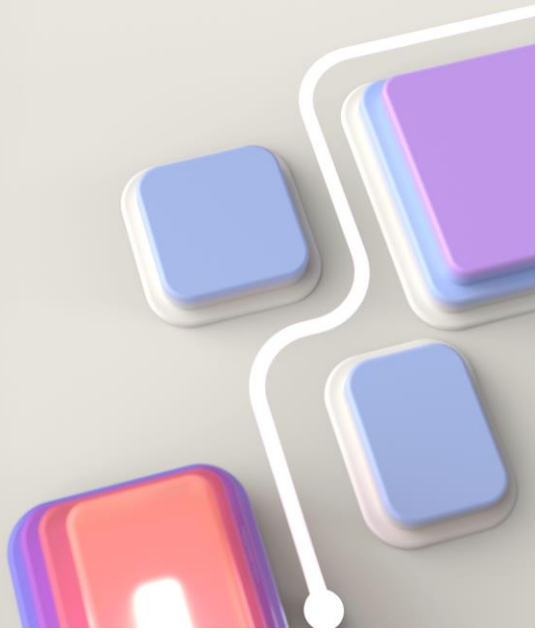
- Add headers
- Add an image
- Add a code example
- Make a task list
- Merge your pull request

Start course

© Copyright Microsoft Corporation. All rights reserved.

53

Module 9: Contribute to an open-source project on GitHub



© Copyright Microsoft Corporation. All rights reserved.

54

© Microsoft Corporation

How to create a project

Most projects will have these documents at the top level of the repository:

- **LICENSE:** The project must contain an open-source license. If the project doesn't have a license, it's not open source.
- **README:** File that serves as the welcome page for the project. It generally provides information on how to get started using the project. It's also common for it to add information on how to engage with the community.
- **CONTRIBUTING:** This document provides guidance on how to contribute to the project. It usually describes how the contribution process works, and includes details on how to set up your development environment.
- **CODE_OF_CONDUCT:** Sets ground rules for community members. By doing so, it helps make the community a safe and welcoming environment for all.

55

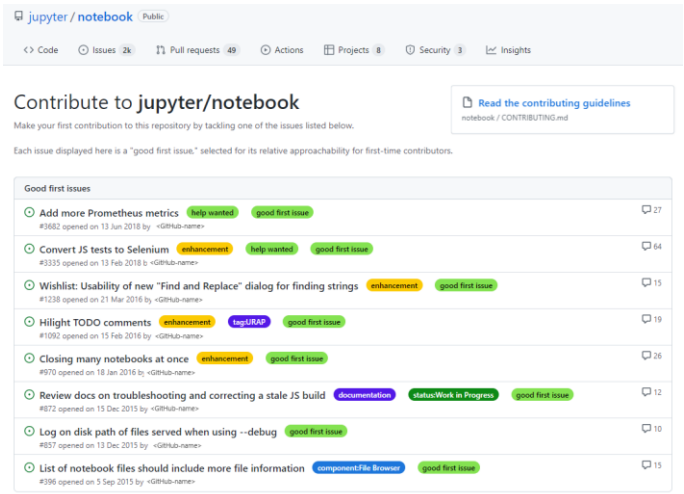
How to create a project (cont.)



Perhaps you've already identified something to work on, such as fixing broken links or updating the docs.

A good way to find issues to help with is by visiting the project's **/contribute** URL.

For example: https://github.com/jupyter/notebook/contribute .



© Copyright Microsoft Corporation. All rights reserved.

56

© Microsoft Corporation

Contribute to an open-source repository

A pull request includes the following activities:

Fork the Repository
Select the Fork button on the GitHub project page to create a copy of the repository on your GitHub account.

Clone the Repository
Copy the repository URL and use the git clone <REPOSITORY_URL> command in your terminal to create a local copy.

Create a New Branch (Optional)
Use git checkout -b <BRANCH_NAME> to create a new branch for your changes, allowing for separate contributions.

Make and Commit Changes
Make the desired changes, then use git add . and git commit -m "<COMMIT_MESSAGE>" to stage and commit them.

Push Changes to GitHub
Push your changes to the remote repository with git push --set-upstream origin <BRANCH_NAME>.

Create a Pull Request
Open your project fork on GitHub, select the Compare & pull request button, fill in the title and description, and create the pull request.

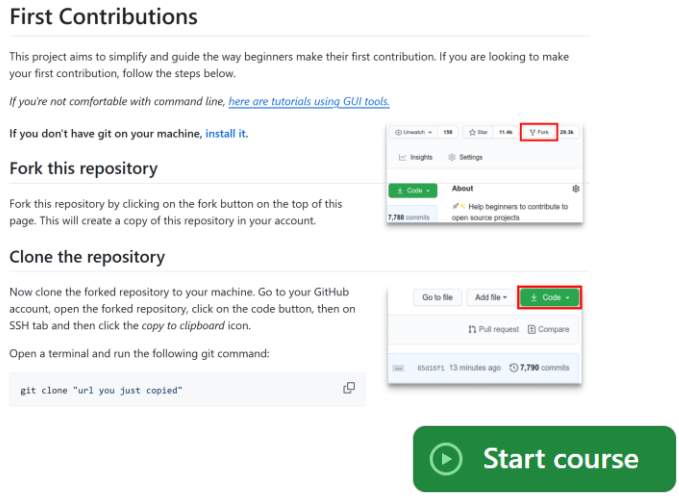
57

Exercise - Create your first pull request



Using the First Contributions project, you practice forking, cloning, and submitting a pull request.

It has guides for both using the command line and several graphical user interfaces (GUIs).



© Copyright Microsoft Corporation. All rights reserved.

58

© Microsoft Corporation

Module 10: Manage an InnerSource program by using GitHub



© Copyright Microsoft Corporation. All rights reserved.

59

How to manage a successful InnerSource program

Why use InnerSource?
- InnerSource applies open-source patterns to projects with a limited audience.
- It creates an open-source-like environment behind a company's firewall for employees.

InnerSource Benefits
- Developers can view and reuse code from other teams, enhancing productivity and understanding project direction.
- Teams can propose changes or fork projects to meet their needs, facilitating collaboration.

Setting Up an InnerSource Program
- Configure repositories as public, internal, or private based on access needs.
- Assign permissions (read, triage, write, maintain, admin) to control access and contributions.

Creating Discoverable Repositories
- Use descriptive names and concise descriptions to make repositories easy to find.
- Include README.md files to communicate project expectations and manage contributions.

Managing Projects on GitHub
- CONTRIBUTING.md: Use this file to explain contribution policies and encourage adherence to project conventions.
- CODEOWNERS: Define individuals or teams responsible for reviewing code modifications.

Creating Issue and Pull Request Templates
- Templates: Provide initial description text for new issues and pull requests to streamline.
- Workflows: Specify workflows for branches, pull requests, and releases for external contributors.

60

© Microsoft Corporation

Exercise - InnerSource fundamentals

The InnerSource guide is an expanded checklist of items to include in your GitHub repositories for the following categories:

- Team
- Repository
- Project
- Developers



Navigate to GitHub's InnerSource guide

© Copyright Microsoft Corporation. All rights reserved.

61

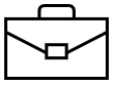
Module 11: Maintain a secure repository by using GitHub best practices

© Copyright Microsoft Corporation. All rights reserved.

62

How to maintain a secure GitHub repository





Benefits of DevOps Practices

- **Integration of Security Testing**: DevOps practices make it easier to integrate security testing into the pipeline, making it a part of daily processes.
- **Developer-Centric Security: S**hifting left focuses on making security more developer-centric, providing feedback early and reducing overall development time.

Importance of Integrating Security Early

- **Secure Development Lifecycle:** Security must be integrated at every stage of the software development lifecycle.
- **Shift Left:** Moving security processes earlier in the development lifecycle reduces mistakes and speeds up development.

GitHub Security Advisories

- **Private Collaboration:** Maintainers can privately discuss and fix security vulnerabilities before publicly disclosing them.
- **Community Updates:** Published advisories help the community update dependencies and understand the impact of vulnerabilities.

© Copyright Microsoft Corporation. All rights reserved.

63

How to maintain a secure GitHub repository (cont.)

Communicate a Security Policy with SECURITY.md

- Provides guidance for responsible disclosure of security issues, speeding up resolution.
- Place the SECURITY.md file in the root of the repository for easy access.

GitHub Security Advisories

- Private Discussions: Allows maintainers to privately discuss and fix security vulnerabilities.
- Public Disclosure: Maintainers can publish advisories to inform the community.

Keep Sensitive Files Out with .gitignore

- Prevents accidental commits of sensitive data by ignoring specified files and patterns.
- Include common patterns like build results and user-specific files in .gitignore.

Remove Sensitive Data from a Repository

- Assume any committed data is compromised; simply overwriting commits is not enough.
- Follow GitHub's guide to thoroughly remove sensitive data from the repository.

Branch Protection Rules

- Create rules to enforce workflows, such as requiring reviews or passing status checks.
- Use rules to run builds, linters, and automated tests to ensure code quality.

Define Workflows

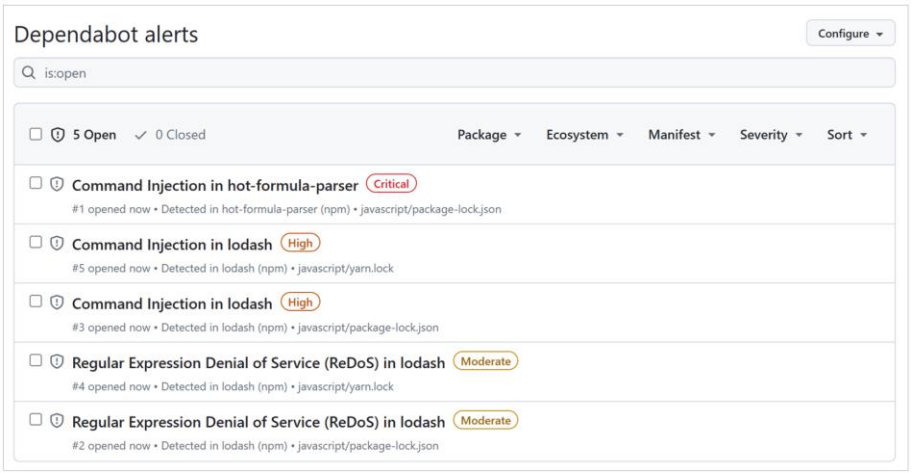
- Specify workflows for branches, pull requests, and releases to guide contributors.
- Clearly communicate the workflow strategy to all contributors.

64

© Microsoft Corporation

Automated security

- **Dependabot Alerts** monitor dependency graphs and cross-references with known vulnerability lists to alert projects of risks.
- **Input Source** utilizes GitHub Security Advisories for analysis and risk detection.

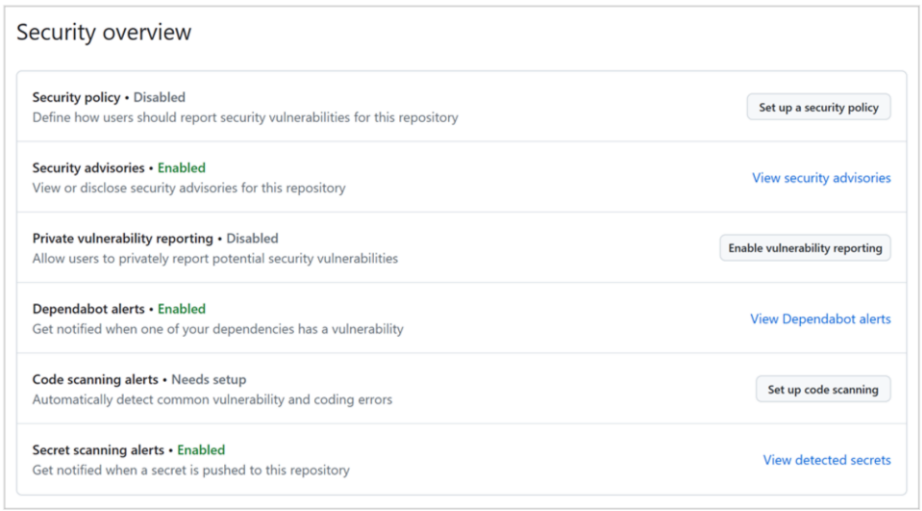


© Copyright Microsoft Corporation. All rights reserved.

65

Automated security (cont.)

- **Code Scanning** helps find, triage, and prioritize fixes for security vulnerabilities and prevents new security issues.
- **CodeQL Integration** allows querying code as data for custom or community-maintained queries.



66

© Microsoft Corporation

Exercise - Secure your repository's supply chain



This exercise is a challenge based on content covered in this module, and there could be more than one way to successfully complete the exercise.

If you get stuck, revisit previous content in this module or navigate to some of the other resources provided.

In this exercise, you will explore:

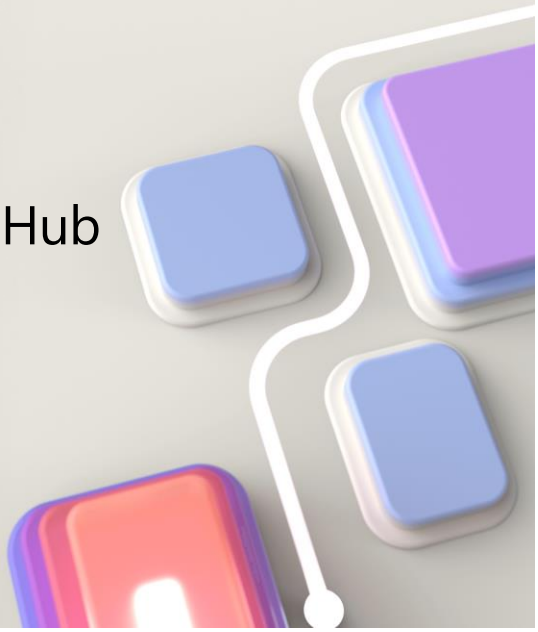
- Dependency graph
- Dependency alerts
- Dependency security updates
- Dependency versions updates

Start the exercise on GitHub

© Copyright Microsoft Corporation. All rights reserved.

67

Module 12: Introduction to GitHub administration



© Copyright Microsoft Corporation. All rights reserved.

68

© Microsoft Corporation

What is GitHub administration?





Administration at Team Level

- **Team Creation and Management:** Create teams with cascading permissions, sync with identity providers like Microsoft Entra ID, and manage team members and outside collaborators.
- **Best Practices:** Reflect company hierarchy with nested teams, streamline PR reviews with interest-based teams, and enable team synchronization to reduce manual updates.

Administration at Organization Level

- **Organization Management:** Invite and remove members, organize users into teams, manage repository permissions, and set up security and billing.
- **Best Practices:** Set up a single organization to avoid duplication, reduce errors, and manage costs effectively.

Administration at Enterprise Level

- **Enterprise Management:** Enable SAML single sign-on, add or remove organizations, manage billing, and set up enterprise-wide policies.
- **Best Practices:** Use custom scripts for extracting information and applying changes across the enterprise.

© Copyright Microsoft Corporation. All rights reserved.

69

How does GitHub authentication work?

Authentication Methods

- **Username and Password:** Basic authentication is risky for sensitive information; alternatives are recommended.
- **Personal Access Tokens (PATs):** Tokens generated via GitHub settings for API or command line authentication, tied to specific permissions.

SSH and Deploy Keys

- **SSH Keys:** Authenticate to remote servers without supplying username and token for every interaction.
- **Deploy Keys:** Grant access to a single repository, with the public key attached to the repository and the private key on the user's server.

Additional Security Options

- **Two-Factor Authentication (2FA):** Adds an extra layer of security by requiring a code from a mobile device or SMS in addition to the password.
- **SAML Single Sign-On (SSO):** Allows centralized management of user identities and secure access to organization resources via an IdP.

Best Practices

- **Team Synchronization:** Sync GitHub teams with identity providers to automate updates and reduce tasks.
- **Security Policies:** Enforce security policies at the organization and enterprise levels to protect resources.

70

© Microsoft Corporation

How do GitHub organization and permissions work?

You can customize access to a given repository by assigning permissions. There are five repository-level permissions:

- **Read:** Recommended for non-code contributors who want to view or discuss your project. This level is good for anyone to view the content within the repository but doesn't need to actually make contributions or changes. that needs
- **Triage:** Recommended for contributors who need to proactively manage issues and pull requests without write access. This level could be good for some project managers who manage tracking issues but don't make any changes.
- **Write:** Recommended for contributors who actively push to your project. Write is the standard permission for most developers.
- **Maintain:** Recommended for project managers who need to manage the repository without access to sensitive or destructive actions.
- **Admin:** Recommended for people who need full access to the project, including sensitive and destructive actions like managing security or deleting a repository. These people are repository owners and administrators.

© Copyright Microsoft Corporation. All rights reserved.

71

How do GitHub organization and permissions work? (cont.)

Organization permission levels

There are six levels of permissions at the organizational level.

Permission level	Description
Owner	Organization owners can do everything that organization members can do, and they can add or remove other users to and from the organization. This role should be limited to no less than two people in your organization.
Member	Organization members can create and manage organization repositories and teams.
Moderator	Organization moderators can block and unblock nonmember contributors, set interaction limits, and hide comments in public repositories that the organization owns.
Billing manager	Organization billing managers can view and edit billing information.
Security managers	Organization security managers can manage security alerts and settings across your organization. They can also read permissions for all repositories in the organization.
Outside collaborator	Outside collaborators, such as a consultant or temporary employee, can access one or more organization repositories. They aren't explicit members of the organization.

© Copyright Microsoft Corporation. All rights reserved.

72

© Microsoft Corporation

Module 13: Authenticate and authorize user identities on GitHub



© Copyright Microsoft Corporation. All rights reserved.

73

User identity and access management

Organization Management through SAML SSO

- SAML SSO: Links IdP authorization with access to service providers, allowing single sign-on for all applications.
- Administrator Responsibilities: Authorize user access, perform routine audits, and maintain tightly scoped access.

Access Management with Multiple Organizations

- Group Management: Use organizations to create distinct user groups within a company, such as divisions or project teams.
- Repository Access: Public and internal repositories are accessible to members of other organizations, while private repositories are restricted.

Organization Private Information

- Repository Visibility: Members can create public, private, or internal repositories, with access controlled based on repository type.
- Access Control: Certain organization members have access to internal repositories, ensuring secure collaboration.

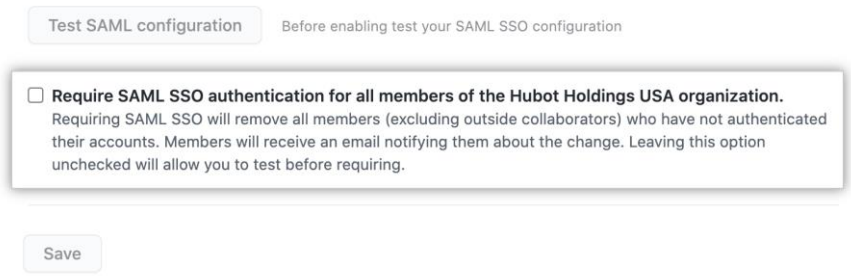
© Copyright Microsoft Corporation. All rights reserved.

74

User authentication (SAML SSO)

Enforce SAML SSO

- **Enforcement :** After enabling SAML SSO, enforce authentication to make it mandatory for all organization members.
- **Steps to Enforce:** Go to Your organizations > Settings > Organization security, and select "Require SAML SSO authentication for all members."

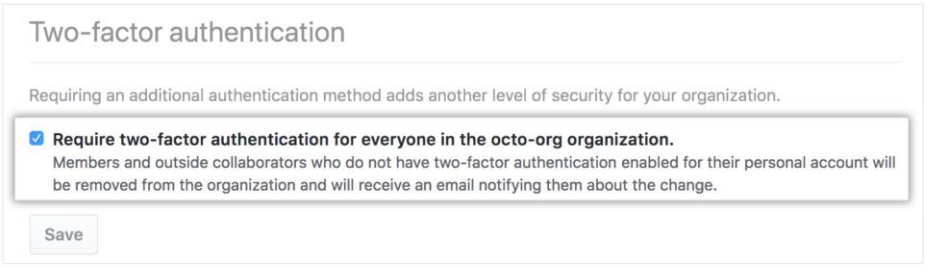


© Copyright Microsoft Corporation. All rights reserved.

75

User authentication (Multifactor authentication)

- **Two-Factor Authentication (2FA):** Requires a secondary form of authentication in addition to the username and password.
- **Implementation:** members, outside collaborators, and billing managers must enable 2FA on their accounts, and administrators should communicate this requirement to the team.



© Copyright Microsoft Corporation. All rights reserved.

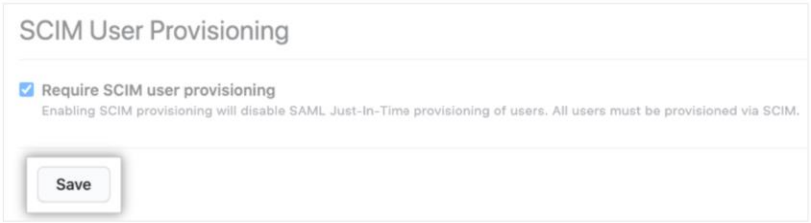
76

© Microsoft Corporation

User authentication (SAML SSO through SCIM)



- **SAML SSO** secures access to GitHub resources, while SCIM automates user identity management, including provisioning and deprovisioning.
- **SCIM** ensures automatic removal of user access when sessions expire, preventing unauthorized access even after session expiration.



© Copyright Microsoft Corporation. All rights reserved.

77

User authentication (SSH key and PAT)



- User lifecycle-management changes in your IdP are automatically reflected in GitHub, with SAML and SCIM ensuring synchronized identities and group memberships.
- Users need to authorize with a PAT or SSH key for API and Git access, and administrators can review these authorizations.



© Copyright Microsoft Corporation. All rights reserved.

78

© Microsoft Corporation

Team synchronization

- Automatically reflects changes made in IdP groups on GitHub, reducing manual updates and custom scripts.
- Allows for efficient onboarding, permission management, and access revocation.
- Requires enabling SAML SSO and SCIM by the IdP Administrator.

Features	Description
Sync Users	Add or remove users from `Teams` in GitHub to keep in sync with Active Directory groups
Sync on new team	Synchronize users when a new team is created
Custom team/group maps	The team `slug` and group name will be matched automatically, unless you define a custom mapping with `syncmap.yml`
Dynamic Config	Utilize a `settings` file to derive Active Directory and GitHub settings

© Copyright Microsoft Corporation. All rights reserved.

79

Module 14: Manage repository changes by using pull requests on GitHub



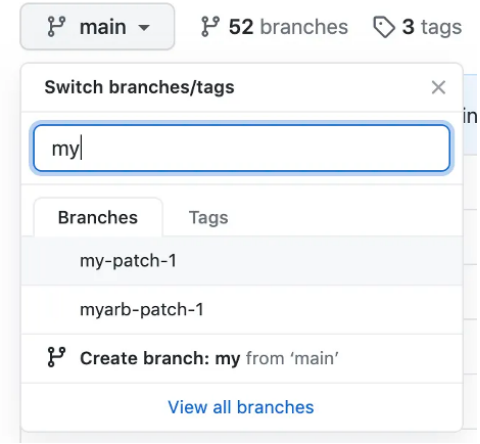
© Copyright Microsoft Corporation. All rights reserved.

80

© Microsoft Corporation

What are pull requests?

- **Purpose of Pull Requests:** Document branch changes and enable stakeholders to review and discuss proposed changes to maintain code quality.
- **Branch Comparison:** The compare branch contains the developer's changes, while the base branch (main branch) is where changes are merged.

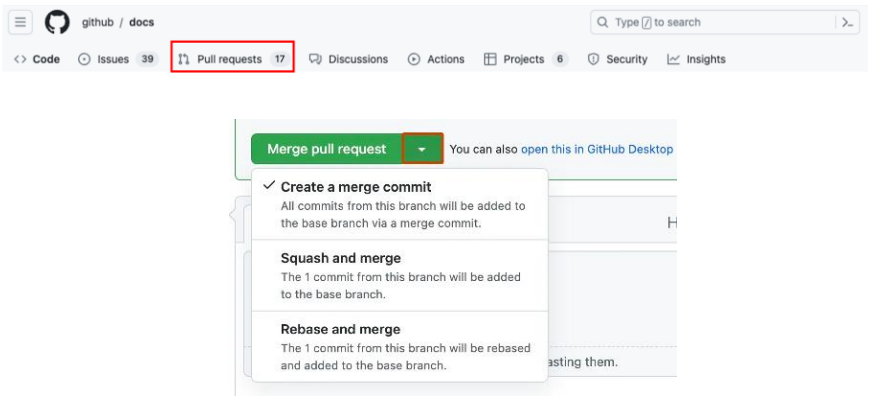


© Copyright Microsoft Corporation. All rights reserved.

81

What are pull requests? (cont.)

- **Merge Options:** You can merge commits into the base branch, squash and merge to combine commits, or rebase and merge to maintain a linear history.
- **Commit Message and Email:** Enter a commit message if prompted and select the appropriate email address for the Git author.



© Copyright Microsoft Corporation. All rights reserved.

82

© Microsoft Corporation

Exercise - Reviewing pull requests



Pull requests are the foundation of teamwork on GitHub.

Pull request reviews give you the ability to work together and discuss changes specific to a pull request by commenting, requesting changes, or approving.

In this exercise, you will:

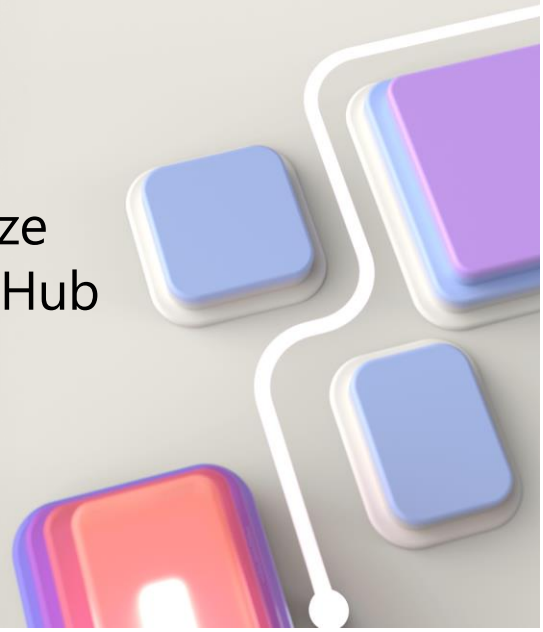
- Open a pull request
- Assign yourself
- Leave a review
- Suggest changes
- Apply changes
- Merge your pull request

Start course

© Copyright Microsoft Corporation. All rights reserved.

83

Module 15: Search and organize repository history by using GitHub



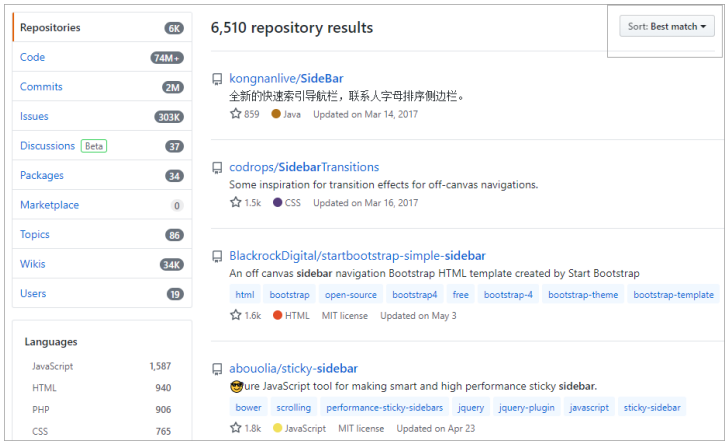
© Copyright Microsoft Corporation. All rights reserved.

84

© Microsoft Corporation

How to search and organize repository history by using GitHub

- **Search Methods:** Use global search for comprehensive results across all of GitHub, and scoped search for specific repository tabs.
- **Starting Point:** Search the project's repository for key terms like "sidebar" to find relevant history and context for fixing issues.

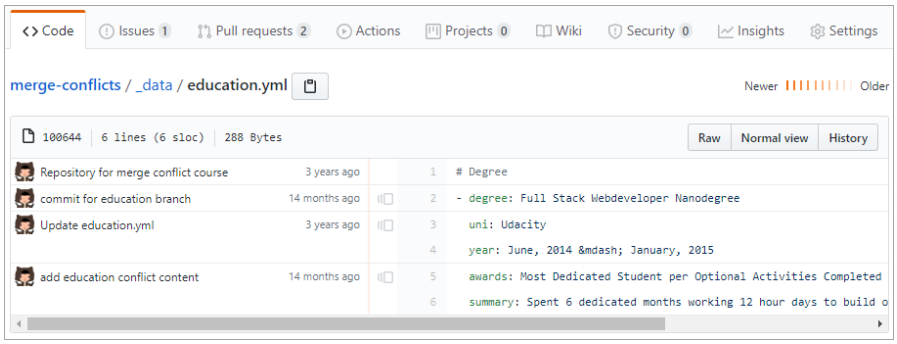


© Copyright Microsoft Corporation. All rights reserved.

85

How to search and organize repository history by using GitHub (Blame command)

- **Git Blame Command:** Displays the commit history for a file, showing who made changes and when, helping to track down contributors for input.
- **GitHub Blame View:** Extends git blame with a robust interface, making it easier to locate subject matter experts by viewing commit history.



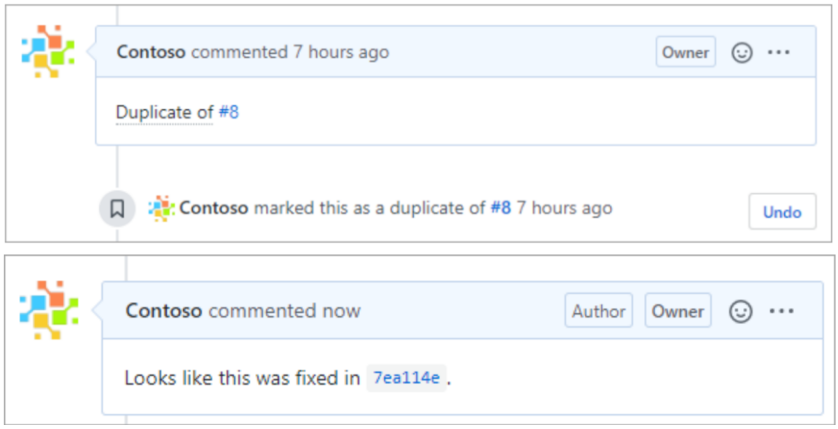
© Copyright Microsoft Corporation. All rights reserved.

86

© Microsoft Corporation

How to search and organize repository history by using GitHub (Autolinked references)

- **Autolinked References:** GitHub offers shorthand syntax to automatically create links for issues and commits, making it easier to cross-link items.
- **Manual Linking:** Use the interface to manually link pull requests or projects to issues, enhancing collaboration and context sharing.



© Copyright Microsoft Corporation. All rights reserved.

87

Exercise - Connect the dots in a GitHub repository

What you'll learn:
- Find relevant issues and pull requests.
- Search history to find context.
- Make connections within GitHub to help others find things.

What you'll build: Repository with existing commits, duplicated issues, and a content defect to be fixed.

In this exercise, you will explore:

- Resolve a duplicate issue.
- Find a commit in history.
- Fix a broken sidebar.

Start course

© Copyright Microsoft Corporation. All rights reserved.

88

© Microsoft Corporation

Module 16: Using GitHub Copilot with Python



© Copyright Microsoft Corporation. All rights reserved.

89

What is GitHub Copilot?

How GitHub Copilot Works

- **AI Assistant:** GitHub Copilot generates code and more based on natural language prompts typed within your IDE.
- **Prompts:** A natural language prompt that can generate code for you to use looks like:

```
# Create a web API using FastAPI
with a route to products
```

Recognizing Prompts

- **Comment in Code File:** Copilot recognizes prompts typed as comments in code files (e.g., .py, .js).
- **Markdown File:** Typing text in a markdown file and waiting a few seconds will also prompt Copilot to respond.

Accepting Suggestions

- **Gray Code:** *Suggestions* appear as gray code; press the **Tab** key to accept.
- **Multiple Suggestions:** Use **Ctrl + Enter** to cycle through and select the most appropriate suggestion.

© Copyright Microsoft Corporation. All rights reserved.

90

Use GitHub Copilot with Python



Developing with GitHub Copilot

- **Continuous Updates:** Ensure code is fresh, fix bugs, and add new features using GitHub Copilot and Copilot Chat.
- **Interactive Chat:** Use Copilot Chat to ask and receive answers to code-related questions.



Prompt Engineering

- **Creating Prompts:** Build prompts with clear instructions to generate specific responses from Copilot.
- **Effective Prompts:** Crafting specific and clear prompts ensures high-quality output from Copilot.



Best Practices for Using Copilot

- **Simple to Elaborate:** Start with simple prompts and gradually add more details for specific suggestions.
- **Cycle Suggestions:** Use **Ctrl+Enter** to cycle through suggestions and select the best output.

© Copyright Microsoft Corporation. All rights reserved.

91

::: The End :::



92

© Microsoft Corporation